

Flight Ticket Booking Management System

25 SQL Questions & Answers per Topic (150 total)

All queries run against the flight_booking_system schema (airports, customers, flights, seats, bookings, payments) and its views/procedures/function defined in create_tables.sql.

Contents

1. FLIGHT REPORTS — 25 questions
2. PASSENGER (CUSTOMER) REPORTS — 25 questions
3. SEAT AVAILABILITY REPORTS — 25 questions
4. BOOKING REPORTS — 25 questions
5. CANCELLATION REPORTS — 25 questions
6. REVENUE REPORTS — 25 questions

1. FLIGHT REPORTS

Q1. List all flights with flight number, departure and arrival time.

Answer:

```
SELECT flight_number, departure_time, arrival_time
FROM flights
ORDER BY departure_time;
```

Plain SELECT with ORDER BY.

Q2. List all flights operated under the 'AI' airline prefix.

Answer:

```
SELECT flight_number, departure_time
FROM flights
WHERE flight_number LIKE 'AI%';
```

LIKE pattern-matching on a prefix.

Q3. Find all flights departing from Chennai.

Answer:

```
SELECT f.flight_number, f.departure_time
FROM flights f
JOIN airports a ON a.airport_id = f.source_airport_id
WHERE a.city = 'Chennai';
```

JOIN to resolve the source_airport_id to a city name.

Q4. Find all flights arriving into Mumbai.

Answer:

```
SELECT f.flight_number, f.arrival_time
FROM flights f
JOIN airports a ON a.airport_id = f.destination_airport_id
WHERE a.city = 'Mumbai';
```

Same pattern, joined on the destination airport instead.

Q5. List all flights that are currently Delayed.

Answer:

```
SELECT flight_number, departure_time, flight_status
FROM flights
WHERE flight_status = 'Delayed';
```

Filter on the flight_status ENUM.

Q6. Count the number of flights in each status category.

Answer:

```
SELECT flight_status, COUNT(*) AS total
FROM flights
GROUP BY flight_status;
```

GROUP BY with COUNT(*)

Q7. Find the number of flights scheduled on each calendar date.

Answer:

```
SELECT DATE(departure_time) AS flight_date, COUNT(*) AS num_flights
FROM flights
GROUP BY DATE(departure_time)
ORDER BY flight_date;
```

DATE() strips the time portion so flights group by day.

Q8. Find the airport with the most outgoing (departing) flights.

Answer:

```
SELECT a.airport_name, a.city, COUNT(*) AS departures
FROM flights f
JOIN airports a ON a.airport_id = f.source_airport_id
GROUP BY a.airport_id, a.airport_name, a.city
ORDER BY departures DESC
LIMIT 1;
```

Aggregate departures per source airport and take the top row.

Q9. Find the airport with the most incoming (arriving) flights.

Answer:

```
SELECT a.airport_name, a.city, COUNT(*) AS arrivals
FROM flights f
JOIN airports a ON a.airport_id = f.destination_airport_id
GROUP BY a.airport_id, a.airport_name, a.city
ORDER BY arrivals DESC
LIMIT 1;
```

Mirror of the previous query, joined on destination instead.

Q10. Find the average flight duration in minutes across all flights.

Answer:

```
SELECT ROUND(AVG(TIMESTAMPDIFF(MINUTE, departure_time, arrival_time)), 1) AS avg_duration_minutes
FROM flights;
```

TIMESTAMPDIFF computes the per-row duration; AVG() aggregates it.

Q11. Find the single longest flight by duration.

Answer:

```
SELECT flight_number, TIMESTAMPDIFF(MINUTE, departure_time, arrival_time) AS duration_minutes
FROM flights
ORDER BY duration_minutes DESC
LIMIT 1;
```

Order the computed duration descending and take the first row.

Q12. Find the single shortest flight by duration.

Answer:

```
SELECT flight_number, TIMESTAMPDIFF(MINUTE, departure_time, arrival_time) AS duration_minutes
FROM flights
ORDER BY duration_minutes ASC
LIMIT 1;
```

Same as above with ascending order.

Q13. List all flights with a duration longer than 8 hours.

Answer:

```
SELECT flight_number, TIMESTAMPDIFF(MINUTE, departure_time, arrival_time) / 60.0 AS duration_hours
FROM flights
HAVING duration_hours > 8
ORDER BY duration_hours DESC;
```

HAVING filters on a computed alias (MySQL allows this even without GROUP BY).

Q14. Find the number of distinct source-to-destination routes.

Answer:

```
SELECT COUNT(DISTINCT source_airport_id, destination_airport_id) AS distinct_routes
FROM flights;
```

COUNT(DISTINCT col1, col2) counts unique combinations of both columns.

Q15. Verify no flight has the same source and destination airport.

Answer:

```
SELECT COUNT(*) AS invalid_routes
FROM flights
WHERE source_airport_id = destination_airport_id;
```

Should always return 0 -- the chk_flights_route CHECK constraint enforces this at insert time.

Q16. Find the number of flights scheduled in each calendar month.

Answer:

```
SELECT DATE_FORMAT(departure_time, '%Y-%m') AS flight_month, COUNT(*) AS num_flights
FROM flights
GROUP BY flight_month
ORDER BY flight_month;
```

DATE_FORMAT buckets departure_time into a year-month string.

Q17. Find flights departing tomorrow.

Answer:

```
SELECT flight_number, departure_time
FROM flights
WHERE DATE(departure_time) = DATE(NOW() + INTERVAL 1 DAY);
```

INTERVAL arithmetic on NOW() computes tomorrow's date for comparison.

Q18. List the top 5 busiest routes by number of flights operated.

Answer:

```
SELECT src.city AS source_city, dst.city AS destination_city, COUNT(*) AS num_flights
FROM flights f
JOIN airports src ON src.airport_id = f.source_airport_id
JOIN airports dst ON dst.airport_id = f.destination_airport_id
GROUP BY src.city, dst.city
ORDER BY num_flights DESC
LIMIT 5;
```

Two joins resolve both airport ends before grouping by the route.

Q19. List all cancelled flights along with their source and destination cities.

Answer:

```
SELECT f.flight_number, src.city AS source_city, dst.city AS destination_city
FROM flights f
JOIN airports src ON src.airport_id = f.source_airport_id
JOIN airports dst ON dst.airport_id = f.destination_airport_id
WHERE f.flight_status = 'Cancelled';
```

Filter after joining so the cities are available in the result set.

Q20. Find the average number of flights operated per source airport.

Answer:

```
SELECT ROUND(AVG(cnt), 2) AS avg_flights_per_airport
FROM (
    SELECT source_airport_id, COUNT(*) AS cnt
    FROM flights
    GROUP BY source_airport_id
) per_airport;
```

A derived table first counts flights per airport, then the outer query averages those counts.

Q21. Find flights whose departure falls on a weekend (Saturday or Sunday).

Answer:

```
SELECT flight_number, departure_time, DAYNAME(departure_time) AS day_of_week
FROM flights
WHERE DAYOFWEEK(departure_time) IN (1, 7);
```

DAYOFWEEK() returns 1 for Sunday and 7 for Saturday in MySQL.

Q22. Find routes that are operated in both directions (A->B and B->A).

Answer:

```
SELECT DISTINCT f1.source_airport_id, f1.destination_airport_id
FROM flights f1
JOIN flights f2
  ON f1.source_airport_id = f2.destination_airport_id
 AND f1.destination_airport_id = f2.source_airport_id;
```

A self-join matches each route against its reverse route.

Q23. Find the flight with the earliest departure time overall.

Answer:

```
SELECT flight_number, departure_time
FROM flights
ORDER BY departure_time ASC
LIMIT 1;
```

Simple ORDER BY + LIMIT 1.

Q24. Find the flight with the latest arrival time overall.

Answer:

```
SELECT flight_number, arrival_time
FROM flights
ORDER BY arrival_time DESC
LIMIT 1;
```

Same pattern on arrival_time, descending.

Q25. List each flight's duration in minutes using the fn_flight_duration_minutes() function.

Answer:

```
SELECT flight_id, flight_number, fn_flight_duration_minutes(flight_id) AS duration_minutes
FROM flights
ORDER BY duration_minutes DESC;
```

Calls the user-defined function created in create_tables.sql once per row.

2. PASSENGER (CUSTOMER) REPORTS

Q1. List all customers with their name, phone and email.

Answer:

```
SELECT customer_name, phone, email
FROM customers;
```

Plain SELECT.

Q2. Find a customer's details by their email address.

Answer:

```
SELECT customer_id, customer_name, phone
FROM customers
WHERE email = 'jane.doe1@example.com';
```

Direct WHERE lookup on a UNIQUE column.

Q3. List customers whose phone numbers start with '+91-98'.

Answer:

```
SELECT customer_name, phone
FROM customers
WHERE phone LIKE '+91-98%';
```

LIKE with a wildcard suffix.

Q4. Count the total number of registered customers.

Answer:

```
SELECT COUNT(*) AS total_customers
FROM customers;
```

COUNT(*) over the whole table.

Q5. Find customers registered in the last 30 days (relative to the most recent signup).

Answer:

```
SELECT customer_name, created_at
FROM customers
WHERE created_at >= (SELECT DATE_SUB(MAX(created_at), INTERVAL 30 DAY) FROM customers)
ORDER BY created_at DESC;
```

A scalar subquery anchors 'last 30 days' to the latest created_at in the data.

Q6. Find customers who have never made a booking.

Answer:

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN bookings b ON b.customer_id = c.customer_id
WHERE b.booking_id IS NULL;
```

LEFT JOIN + IS NULL finds parent rows with no matching child.

Q7. Find customers who have made exactly one booking.

Answer:

```
SELECT c.customer_id, c.customer_name, COUNT(*) AS total_bookings
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(*) = 1;
```

GROUP BY + HAVING COUNT(*) = 1 isolates single-booking customers.

Q8. Find customers who have made more than 5 bookings.

Answer:

```
SELECT c.customer_id, c.customer_name, COUNT(*) AS total_bookings
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(*) > 5
ORDER BY total_bookings DESC;
```

Same pattern with a higher HAVING threshold.

Q9. Find each customer's total number of bookings (including 0 for those with none).

Answer:

```
SELECT c.customer_id, c.customer_name, COUNT(b.booking_id) AS total_bookings
FROM customers c
LEFT JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_bookings DESC;
```

LEFT JOIN keeps customers with zero bookings; COUNT(b.booking_id) ignores NULL rows correctly.

Q10. Find each customer's number of Confirmed bookings only.

Answer:

```
SELECT c.customer_id, c.customer_name,
       SUM(CASE WHEN b.booking_status = 'Confirmed' THEN 1 ELSE 0 END) AS confirmed_bookings
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name;
```

Conditional aggregation counts only rows matching the status.

Q11. Find the top 10 customers by number of bookings.

Answer:

```
SELECT c.customer_id, c.customer_name, COUNT(*) AS total_bookings
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_bookings DESC
LIMIT 10;
```

GROUP BY + ORDER BY + LIMIT for a leaderboard.

Q12. Find the top 10 customers by total amount spent.

Answer:

```
SELECT c.customer_id, c.customer_name, SUM(p.amount) AS total_spent
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC
LIMIT 10;
```

Joins through bookings to payments, summing only completed payments.

Q13. Find customers who have never made a successful payment.

Answer:

```
SELECT DISTINCT c.customer_id, c.customer_name
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
LEFT JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
WHERE p.payment_id IS NULL;
```

LEFT JOIN with a status filter in the ON clause, then check for no match.

Q14. Find the average amount spent per customer.

Answer:

```
SELECT ROUND(AVG(customer_total), 2) AS avg_spend_per_customer
FROM (
    SELECT b.customer_id, SUM(p.amount) AS customer_total
    FROM bookings b
    JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
    GROUP BY b.customer_id
) per_customer;
```

A derived table totals spend per customer, then the outer query averages it.

Q15. Find customers who booked a flight to Delhi.

Answer:

```
SELECT DISTINCT c.customer_id, c.customer_name
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
JOIN flights f ON f.flight_id = b.flight_id
JOIN airports a ON a.airport_id = f.destination_airport_id
WHERE a.city = 'Delhi';
```

Chain of joins from customer through booking, flight, to destination airport.

Q16. Find the customer who has flown the most (most bookings, any status).

Answer:

```
SELECT c.customer_id, c.customer_name, COUNT(*) AS flight_count
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY flight_count DESC
LIMIT 1;
```

Same leaderboard pattern, limited to the single top row.

Q17. List each customer's most recent booking date.

Answer:

```
SELECT c.customer_id, c.customer_name, MAX(b.booking_date) AS last_booking_date
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name;
```

MAX() aggregate per customer.

Q18. Find customers who have at least one cancelled booking.

Answer:

```
SELECT DISTINCT c.customer_id, c.customer_name
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
WHERE b.booking_status = 'Cancelled';
```

DISTINCT avoids duplicate rows if a customer has multiple cancellations.

Q19. Find customers who have both a Confirmed and a Cancelled booking.

Answer:

```
SELECT c.customer_id, c.customer_name
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(b.booking_status = 'Confirmed') > 0
AND SUM(b.booking_status = 'Cancelled') > 0;
```

In MySQL a boolean expression evaluates to 1/0, so SUM() can count matching rows directly.

Q20. Count customers grouped by their email domain.

Answer:

```
SELECT SUBSTRING_INDEX(email, '@', -1) AS email_domain, COUNT(*) AS total
FROM customers
GROUP BY email_domain
ORDER BY total DESC;
```

SUBSTRING_INDEX(email, '@', -1) extracts everything after the '@'.

Q21. Find the customer with the highest single payment amount.

Answer:

```
SELECT c.customer_id, c.customer_name, p.amount
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
JOIN payments p ON p.booking_id = b.booking_id
ORDER BY p.amount DESC
LIMIT 1;
```

Sort individual payments (not aggregated) and take the top one.

Q22. Rank all customers by total spend using a window function.

Answer:

```
SELECT customer_id, customer_name, total_spent,
       RANK() OVER (ORDER BY total_spent DESC) AS spend_rank
FROM (
    SELECT c.customer_id, c.customer_name, SUM(p.amount) AS total_spent
    FROM customers c
    JOIN bookings b ON b.customer_id = c.customer_id
    JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
    GROUP BY c.customer_id, c.customer_name
) spend;
```

RANK() OVER (ORDER BY ...) assigns a competition rank over the aggregated spend.

Q23. Find customers whose bookings are exclusively Economy class.

Answer:

```
SELECT c.customer_id, c.customer_name
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
JOIN seats s ON s.seat_id = b.seat_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(s.seat_class <> 'Economy') = 0;
```

HAVING checks that the count of non-Economy bookings is zero.

Q24. Find customers who booked the same flight more than once.

Answer:

```
SELECT c.customer_id, c.customer_name, b.flight_id, COUNT(*) AS bookings_on_flight
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name, b.flight_id
HAVING COUNT(*) > 1;
```

Grouping by both customer and flight surfaces repeat bookings on the same flight.

Q25. Find each customer's first (earliest) booking date.

Answer:

```
SELECT c.customer_id, c.customer_name, MIN(b.booking_date) AS first_booking_date
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name;
```

MIN() aggregate mirrors the earlier MAX() query.

3. SEAT AVAILABILITY REPORTS

Q1. List all seats belonging to flight_id = 1.

Answer:

```
SELECT seat_number, seat_class, seat_price, seat_status
FROM seats
WHERE flight_id = 1
ORDER BY seat_number;
```

Basic filter + sort.

Q2. Find all Available seats on flight_id = 1.

Answer:

```
SELECT seat_number, seat_class, seat_price
FROM seats
WHERE flight_id = 1 AND seat_status = 'Available';
```

Two-condition WHERE clause.

Q3. Find all Booked seats on flight_id = 1.

Answer:

```
SELECT seat_number, seat_class
FROM seats
WHERE flight_id = 1 AND seat_status = 'Booked';
```

Same pattern with the opposite status.

Q4. Count seats by class for flight_id = 1.

Answer:

```
SELECT seat_class, COUNT(*) AS total_seats
FROM seats
WHERE flight_id = 1
GROUP BY seat_class;
```

GROUP BY restricted to a single flight.

Q5. Find the total number of seats across all flights.

Answer:

```
SELECT COUNT(*) AS total_seats
FROM seats;
```

COUNT(*) over the whole table.

Q6. Find the total number of Available seats system-wide.

Answer:

```
SELECT COUNT(*) AS available_seats
FROM seats
WHERE seat_status = 'Available';
```

Simple filtered COUNT.

Q7. Find the seat availability percentage for every flight.

Answer:

```
SELECT flight_id,
       ROUND(SUM(seat_status = 'Available') * 100.0 / COUNT(*), 2) AS availability_pct
FROM seats
GROUP BY flight_id
ORDER BY availability_pct DESC;
```

SUM() on a boolean expression counts matching rows; dividing by COUNT(*) gives a percentage.

Q8. Find flights that are completely fully booked (0 available seats).

Answer:

```
SELECT flight_id, COUNT(*) AS total_seats
FROM seats
GROUP BY flight_id
HAVING SUM(seat_status = 'Available') = 0;
```

HAVING filters groups where the available-seat count is zero.

Q9. Find flights with more than 50% of seats still available.

Answer:

```
SELECT flight_id,
       ROUND(SUM(seat_status = 'Available') * 100.0 / COUNT(*), 2) AS availability_pct
FROM seats
GROUP BY flight_id
HAVING availability_pct > 50
ORDER BY availability_pct DESC;
```

Builds on the earlier availability percentage query with a HAVING threshold.

Q10. Find the cheapest available seat on flight_id = 1.

Answer:

```
SELECT seat_number, seat_class, seat_price
FROM seats
WHERE flight_id = 1 AND seat_status = 'Available'
ORDER BY seat_price ASC
LIMIT 1;
```

ORDER BY price ascending + LIMIT 1.

Q11. Find the average seat price for each seat class.

Answer:

```
SELECT seat_class, ROUND(AVG(seat_price), 2) AS avg_price
FROM seats
GROUP BY seat_class
ORDER BY avg_price DESC;
```

AVG() grouped by class.

Q12. List seat classes with their minimum and maximum price.

Answer:

```
SELECT seat_class, MIN(seat_price) AS min_price, MAX(seat_price) AS max_price
FROM seats
GROUP BY seat_class;
```

MIN()/MAX() together in one GROUP BY.

Q13. Find the total number of Business class seats available system-wide.

Answer:

```
SELECT COUNT(*) AS available_business_seats
FROM seats
WHERE seat_class = 'Business' AND seat_status = 'Available';
```

Filter on two columns simultaneously.

Q14. Find flights that currently have zero available Business class seats.

Answer:

```
SELECT flight_id
FROM seats
WHERE seat_class = 'Business'
GROUP BY flight_id
HAVING SUM(seat_status = 'Available') = 0;
```

GROUP BY per flight restricted to Business class rows only.

Q15. Find the single most expensive seat in the entire system.

Answer:

```
SELECT flight_id, seat_number, seat_class, seat_price
FROM seats
ORDER BY seat_price DESC
LIMIT 1;
```

Global ORDER BY + LIMIT 1.

Q16. Find the total ticket revenue potential per flight if every seat sold at seat_price.

Answer:

```
SELECT flight_id, SUM(seat_price) AS potential_revenue
FROM seats
GROUP BY flight_id
ORDER BY potential_revenue DESC;
```

SUM() of list price per flight, independent of actual bookings.

Q17. List all seats currently marked as Blocked.

Answer:

```
SELECT flight_id, seat_number, seat_class
FROM seats
WHERE seat_status = 'Blocked';
```

Filter on the third ENUM value.

Q18. Find the percentage share of each seat class across the whole seat inventory.

Answer:

```
SELECT seat_class, COUNT(*) AS total,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM seats), 2) AS pct_of_total
FROM seats
GROUP BY seat_class
ORDER BY pct_of_total DESC;
```

A scalar subquery supplies the grand total used as the percentage denominator.

Q19. Find the average seat price per flight.

Answer:

```
SELECT flight_id, ROUND(AVG(seat_price), 2) AS avg_seat_price
FROM seats
GROUP BY flight_id
ORDER BY avg_seat_price DESC;
```

AVG() grouped by flight_id.

Q20. Find the number of available seats per class, system-wide.

Answer:

```
SELECT seat_class, COUNT(*) AS available_count
FROM seats
WHERE seat_status = 'Available'
GROUP BY seat_class
ORDER BY available_count DESC;
```

Filter then GROUP BY.

Q21. Find flights where Economy is fully booked but other classes still have availability.

Answer:

```
SELECT s.flight_id
FROM seats s
GROUP BY s.flight_id
HAVING SUM(s.seat_class = 'Economy' AND s.seat_status = 'Available') = 0
       AND SUM(s.seat_class <> 'Economy' AND s.seat_status = 'Available') > 0;
```

Two conditional SUM() expressions in the same HAVING clause.

Q22. Count how many seats are in row 'A' (seat_number starts with 'A').

Answer:

```
SELECT COUNT(*) AS row_a_seats
FROM seats
WHERE seat_number LIKE 'A%';
```

LIKE pattern on the seat_number column.

Q23. Rank flights by number of available seats, descending.

Answer:

```
SELECT flight_id, SUM(seat_status = 'Available') AS available_seats,
       RANK() OVER (ORDER BY SUM(seat_status = 'Available') DESC) AS availability_rank
FROM seats
GROUP BY flight_id;
```

A window function ranks the aggregated per-flight availability count.

Q24. Find the total dollar value of all currently available seats.

Answer:

```
SELECT SUM(seat_price) AS available_seat_value
FROM seats
WHERE seat_status = 'Available';
```

SUM() restricted to Available rows.

Q25. Using vw_seat_availability, list flights with less than 10% seats available (any class).

Answer:

```
SELECT flight_id, flight_number,
       SUM(available_seats) AS available_seats,
       SUM(total_seats) AS total_seats,
       ROUND(SUM(available_seats) * 100.0 / SUM(total_seats), 2) AS availability_pct
FROM vw_seat_availability
GROUP BY flight_id, flight_number
HAVING availability_pct < 10
ORDER BY availability_pct;
```

Re-aggregates the pre-built view (which is already grouped by class) up to the flight level.

4. BOOKING REPORTS

Q1. List all bookings with their status.

Answer:

```
SELECT booking_id, customer_id, flight_id, booking_date, booking_status
FROM bookings
ORDER BY booking_date DESC;
```

Plain SELECT + ORDER BY.

Q2. Find full details for a specific booking_id.

Answer:

```
SELECT *
FROM vw_booking_details
WHERE booking_id = 10;
```

Uses the pre-joined vw_booking_details view for a one-line lookup.

Q3. List all bookings made on a specific calendar date.

Answer:

```
SELECT booking_id, customer_id, flight_id
FROM bookings
WHERE DATE(booking_date) = '2025-03-15';
```

DATE() strips the time component for an exact-day match.

Q4. Count the total number of Confirmed bookings.

Answer:

```
SELECT COUNT(*) AS confirmed_bookings
FROM bookings
WHERE booking_status = 'Confirmed';
```

Filtered COUNT(*).

Q5. Count the total number of Pending bookings.

Answer:

```
SELECT COUNT(*) AS pending_bookings
FROM bookings
WHERE booking_status = 'Pending';
```

Same pattern, different status.

Q6. Find bookings made in the last 7 days (relative to the latest booking in the data).

Answer:

```
SELECT booking_id, customer_id, booking_date
FROM bookings
WHERE booking_date >= (SELECT DATE_SUB(MAX(booking_date), INTERVAL 7 DAY) FROM bookings)
ORDER BY booking_date DESC;
```

Scalar subquery anchors the 7-day window to the most recent booking_date.

Q7. Find the single busiest booking day (most bookings placed on the same date).

Answer:

```
SELECT DATE(booking_date) AS booking_day, COUNT(*) AS total
FROM bookings
GROUP BY booking_day
ORDER BY total DESC
LIMIT 1;
```

GROUP BY day, then take the top row.

Q8. Find the monthly booking trend.

Answer:

```
SELECT DATE_FORMAT(booking_date, '%Y-%m') AS booking_month, COUNT(*) AS total_bookings
FROM bookings
GROUP BY booking_month
ORDER BY booking_month;
```

DATE_FORMAT buckets bookings into year-month groups.

Q9. List all bookings for a specific flight_id.

Answer:

```
SELECT booking_id, customer_id, seat_id, booking_status
FROM bookings
WHERE flight_id = 5;
```

Basic WHERE filter.

Q10. List a specific customer's bookings with flight number and seat number.

Answer:

```
SELECT b.booking_id, f.flight_number, s.seat_number, b.booking_status
FROM bookings b
JOIN flights f ON f.flight_id = b.flight_id
JOIN seats s ON s.seat_id = b.seat_id
WHERE b.customer_id = 42;
```

Two joins bring in flight and seat detail for one customer.

Q11. Find the average number of bookings placed per day.

Answer:

```
SELECT ROUND(AVG(daily_count), 2) AS avg_bookings_per_day
FROM (
    SELECT DATE(booking_date) AS d, COUNT(*) AS daily_count
    FROM bookings
    GROUP BY d
) per_day;
```

A derived table counts per day, then the outer query averages those counts.

Q12. Find the percentage breakdown of booking statuses.

Answer:

```
SELECT booking_status, COUNT(*) AS total,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM bookings), 2) AS pct
FROM bookings
GROUP BY booking_status;
```

A scalar subquery supplies the grand total as the percentage denominator.

Q13. Find the number of bookings for Business class seats.

Answer:

```
SELECT COUNT(*) AS business_bookings
FROM bookings b
JOIN seats s ON s.seat_id = b.seat_id
WHERE s.seat_class = 'Business';
```

Join to seats to filter on seat_class.

Q14. Find the number of bookings per seat class.

Answer:

```
SELECT s.seat_class, COUNT(*) AS total_bookings
FROM bookings b
JOIN seats s ON s.seat_id = b.seat_id
GROUP BY s.seat_class
ORDER BY total_bookings DESC;
```

JOIN + GROUP BY on the joined column.

Q15. List the 5 most recent bookings with full details.

Answer:

```
SELECT *
FROM vw_booking_details
ORDER BY booking_id DESC
LIMIT 5;
```

Reuses the vw_booking_details view, ordered and limited.

Q16. Find how many days in advance customers typically book, on average.

Answer:

```
SELECT ROUND(AVG(DATEDIFF(f.departure_time, b.booking_date)), 1) AS avg_days_in_advance
FROM bookings b
JOIN flights f ON f.flight_id = b.flight_id;
```

DATEDIFF() computes the gap in days between booking and departure, per row.

Q17. Find the 10 most 'last-minute' bookings (smallest gap between booking and departure).

Answer:

```
SELECT b.booking_id, DATEDIFF(f.departure_time, b.booking_date) AS days_in_advance
FROM bookings b
JOIN flights f ON f.flight_id = b.flight_id
ORDER BY days_in_advance ASC
LIMIT 10;
```

Same DATEDIFF calculation, sorted ascending to surface the smallest gaps.

Q18. Find bookings that do not yet have an associated payment.

Answer:

```
SELECT b.booking_id, b.customer_id, b.flight_id
FROM bookings b
LEFT JOIN payments p ON p.booking_id = b.booking_id
WHERE p.payment_id IS NULL;
```

LEFT JOIN + IS NULL, the standard 'missing child row' pattern.

Q19. Find customers who booked the same flight more than once.

Answer:

```
SELECT customer_id, flight_id, COUNT(*) AS times_booked
FROM bookings
GROUP BY customer_id, flight_id
HAVING COUNT(*) > 1;
```

GROUP BY a composite key to spot duplicates.

Q20. Find the number of bookings made on each day of the week.

Answer:

```
SELECT DAYNAME(booking_date) AS day_of_week, COUNT(*) AS total
FROM bookings
GROUP BY day_of_week
ORDER BY FIELD(day_of_week, 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday');
```

DAYNAME() extracts the weekday name; FIELD() forces a natural Mon-Sun ordering.

Q21. Number each customer's bookings in chronological order.

Answer:

```
SELECT booking_id, customer_id, booking_date,
       ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY booking_date) AS booking_sequence
FROM bookings;
```

PARTITION BY resets ROW_NUMBER() for every customer.

Q22. Find the booking with the highest paid amount.

Answer:

```
SELECT b.booking_id, p.amount
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id
ORDER BY p.amount DESC
LIMIT 1;
```

JOIN then ORDER BY + LIMIT 1.

Q23. Find Pending bookings older than 2 days (candidates for auto-cancellation).

Answer:

```
SELECT booking_id, customer_id, booking_date
FROM bookings
WHERE booking_status = 'Pending'
      AND booking_date < (SELECT DATE_SUB(MAX(booking_date), INTERVAL 2 DAY) FROM bookings);
```

Combines a status filter with a relative-date subquery.

Q24. Find the running (cumulative) total of bookings over time.

Answer:

```
SELECT booking_month, monthly_bookings,
       SUM(monthly_bookings) OVER (ORDER BY booking_month) AS cumulative_bookings
FROM (
  SELECT DATE_FORMAT(booking_date, '%Y-%m') AS booking_month, COUNT(*) AS monthly_bookings
  FROM bookings
  GROUP BY booking_month
) monthly
ORDER BY booking_month;
```

A window SUM() over an ordered monthly aggregate produces a running total.

Q25. Book a new ticket using the `sp_book_ticket` stored procedure.

Answer:

```
CALL sp_book_ticket(
  1001,      -- p_booking_id
  5,         -- p_customer_id
  12,        -- p_flight_id
  250,       -- p_seat_id (must currently be Available)
  6500.00,   -- p_amount
  'UPI',     -- p_payment_method
);
```

The procedure locks the seat row, checks availability, and inserts the booking + payment in one transaction.

5. CANCELLATION REPORTS

Q1. List all cancelled bookings.

Answer:

```
SELECT booking_id, customer_id, flight_id, booking_date
FROM bookings
WHERE booking_status = 'Cancelled';
```

Basic filter on booking_status.

Q2. Count the total number of cancellations.

Answer:

```
SELECT COUNT(*) AS total_cancellations
FROM bookings
WHERE booking_status = 'Cancelled';
```

Filtered COUNT(*).

Q3. Find the overall cancellation rate as a percentage.

Answer:

```
SELECT ROUND(
    SUM(booking_status = 'Cancelled') * 100.0 / COUNT(*), 2
) AS cancellation_rate_pct
FROM bookings;
```

SUM() on a boolean expression counts cancelled rows; dividing by COUNT(*) gives the rate.

Q4. Find the cancellation rate for every individual flight.

Answer:

```
SELECT flight_id,
    COUNT(*) AS total_bookings,
    SUM(booking_status = 'Cancelled') AS cancellations,
    ROUND(SUM(booking_status = 'Cancelled') * 100.0 / COUNT(*), 2) AS cancellation_pct
FROM bookings
GROUP BY flight_id
ORDER BY cancellation_pct DESC;
```

Same rate calculation, grouped per flight.

Q5. Find the 5 flights with the highest cancellation rate.

Answer:

```
SELECT flight_id,
    ROUND(SUM(booking_status = 'Cancelled') * 100.0 / COUNT(*), 2) AS cancellation_pct
FROM bookings
GROUP BY flight_id
ORDER BY cancellation_pct DESC
LIMIT 5;
```

Builds on the previous query with LIMIT 5.

Q6. Find the customers with the most cancellations.

Answer:

```
SELECT c.customer_id, c.customer_name, COUNT(*) AS cancellations
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
WHERE b.booking_status = 'Cancelled'
GROUP BY c.customer_id, c.customer_name
ORDER BY cancellations DESC
LIMIT 10;
```

Filter then GROUP BY + ORDER BY for a leaderboard.

Q7. Find the total amount refunded to customers.

Answer:

```
SELECT SUM(amount) AS total_refunded
FROM payments
WHERE payment_status = 'Refunded';
```

SUM() restricted to Refunded payments.

Q8. List all refunded payments with customer and flight context.

Answer:

```
SELECT c.customer_name, f.flight_number, p.amount, p.payment_date
FROM payments p
JOIN bookings b ON b.booking_id = p.booking_id
JOIN customers c ON c.customer_id = b.customer_id
JOIN flights f ON f.flight_id = b.flight_id
WHERE p.payment_status = 'Refunded';
```

Chain of joins gives full context to each refund.

Q9. Find the number of cancellations per month.

Answer:

```
SELECT DATE_FORMAT(booking_date, '%Y-%m') AS month, COUNT(*) AS cancellations
FROM bookings
WHERE booking_status = 'Cancelled'
GROUP BY month
ORDER BY month;
```

Filter first, then GROUP BY month.

Q10. Find the average gap between booking and payment date for refunded bookings.

Answer:

```
SELECT ROUND(AVG(DATEDIFF(p.payment_date, b.booking_date)), 1) AS avg_days_to_refund
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id
WHERE b.booking_status = 'Cancelled' AND p.payment_status = 'Refunded';
```

DATEDIFF() measures the days between the original booking and the refund transaction.

Q11. Find the percentage of total revenue lost to refunds.

Answer:

```
SELECT ROUND(
    (SELECT SUM(amount) FROM payments WHERE payment_status = 'Refunded')
    / (SELECT SUM(amount) FROM payments) * 100, 2
) AS pct_revenue_refunded;
```

Two scalar subqueries compute refunded vs. total payment value.

Q12. Find which seat class has the highest number of cancellations.

Answer:

```
SELECT s.seat_class, COUNT(*) AS cancellations
FROM bookings b
JOIN seats s ON s.seat_id = b.seat_id
WHERE b.booking_status = 'Cancelled'
GROUP BY s.seat_class
ORDER BY cancellations DESC;
```

Join to seats, then GROUP BY class after filtering to cancellations.

Q13. Find flights that have never had a cancellation.

Answer:

```
SELECT f.flight_id, f.flight_number
FROM flights f
JOIN bookings b ON b.flight_id = f.flight_id
GROUP BY f.flight_id, f.flight_number
HAVING SUM(b.booking_status = 'Cancelled') = 0;
```

HAVING checks that the cancelled-count for the group is zero.

Q14. Find cancelled bookings whose payment hasn't been marked Refunded yet (data-quality check).

Answer:

```
SELECT b.booking_id, p.payment_status
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id
WHERE b.booking_status = 'Cancelled' AND p.payment_status <> 'Refunded';
```

Flags inconsistent rows where cancellation and refund status disagree.

Q15. Find which day of the week sees the most cancellations.

Answer:

```
SELECT DAYNAME(booking_date) AS day_of_week, COUNT(*) AS cancellations
FROM bookings
WHERE booking_status = 'Cancelled'
GROUP BY day_of_week
ORDER BY cancellations DESC
LIMIT 1;
```

DAYNAME() + GROUP BY, take the top row.

Q16. Rank flights by number of cancellations, descending.

Answer:

```
SELECT flight_id, COUNT(*) AS cancellations,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS cancellation_rank
FROM bookings
WHERE booking_status = 'Cancelled'
GROUP BY flight_id;
```

A window RANK() over the per-flight cancellation count.

Q17. Find the total number of cancellations grouped by source city.

Answer:

```
SELECT src.city AS source_city, COUNT(*) AS cancellations
FROM bookings b
JOIN flights f ON f.flight_id = b.flight_id
JOIN airports src ON src.airport_id = f.source_airport_id
WHERE b.booking_status = 'Cancelled'
GROUP BY src.city
ORDER BY cancellations DESC;
```

Joins through flights to resolve the source city before grouping.

Q18. Find the cumulative number of cancellations over time.

Answer:

```
SELECT month, monthly_cancellations,
       SUM(monthly_cancellations) OVER (ORDER BY month) AS cumulative_cancellations
FROM (
  SELECT DATE_FORMAT(booking_date, '%Y-%m') AS month, COUNT(*) AS monthly_cancellations
  FROM bookings
  WHERE booking_status = 'Cancelled'
  GROUP BY month
) monthly
ORDER BY month;
```

Window SUM() over an ordered monthly aggregate, restricted to cancellations.

Q19. Find customers whose personal cancellation rate exceeds 50%.

Answer:

```
SELECT c.customer_id, c.customer_name,
       ROUND(SUM(b.booking_status = 'Cancelled') * 100.0 / COUNT(*), 2) AS cancellation_pct
FROM customers c
JOIN bookings b ON b.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING cancellation_pct > 50
ORDER BY cancellation_pct DESC;
```

Per-customer version of the earlier flight-level cancellation-rate query.

Q20. Find the single most expensive cancelled booking (largest refund).

Answer:

```
SELECT b.booking_id, c.customer_name, p.amount
FROM bookings b
JOIN customers c ON c.customer_id = b.customer_id
JOIN payments p ON p.booking_id = b.booking_id
WHERE b.booking_status = 'Cancelled'
ORDER BY p.amount DESC
LIMIT 1;
```

Filter, join, then ORDER BY + LIMIT 1.

Q21. List all cancelled bookings for a specific flight with refund status.

Answer:

```
SELECT b.booking_id, c.customer_name, p.amount, p.payment_status
FROM bookings b
JOIN customers c ON c.customer_id = b.customer_id
LEFT JOIN payments p ON p.booking_id = b.booking_id
WHERE b.flight_id = 5 AND b.booking_status = 'Cancelled';
```

Combines a flight filter with the cancellation status filter.

Q22. Compare Confirmed vs. Cancelled counts side by side for each flight.

Answer:

```
SELECT flight_id,
       SUM(booking_status = 'Confirmed') AS confirmed_count,
       SUM(booking_status = 'Cancelled') AS cancelled_count
FROM bookings
GROUP BY flight_id
ORDER BY flight_id;
```

Two conditional SUM() expressions pivot the status counts into columns.

Q23. Find flights whose cancellation rate is above the system-wide average.

Answer:

```
SELECT flight_id, cancellation_pct
FROM (
    SELECT flight_id,
           SUM(booking_status = 'Cancelled') * 100.0 / COUNT(*) AS cancellation_pct
    FROM bookings
    GROUP BY flight_id
) per_flight
WHERE cancellation_pct > (
    SELECT SUM(booking_status = 'Cancelled') * 100.0 / COUNT(*) FROM bookings
)
ORDER BY cancellation_pct DESC;
```

Compares each flight's rate (derived table) against the global rate (scalar subquery).

Q24. Cancel an existing booking using the `sp_cancel_booking` stored procedure.

Answer:

```
CALL sp_cancel_booking(1001);
```

The procedure marks the booking Cancelled, frees the seat, and marks the payment Refunded in one transaction.

Q25. Verify that every Available seat freed by a cancellation matches a Cancelled booking.

Answer:

```
SELECT COUNT(*) AS mismatched_seats
FROM seats s
JOIN bookings b ON b.seat_id = s.seat_id
WHERE s.seat_status = 'Available' AND b.booking_status <> 'Cancelled';
```

A data-integrity check: should return 0 if the seat/booking sync trigger is working correctly.

6. REVENUE REPORTS

Q1. Find the total revenue generated overall (completed payments only).

Answer:

```
SELECT SUM(amount) AS total_revenue
FROM payments
WHERE payment_status = 'Completed';
```

SUM() restricted to Completed payments.

Q2. Find the total revenue generated by each flight.

Answer:

```
SELECT b.flight_id, SUM(p.amount) AS revenue
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id
WHERE p.payment_status = 'Completed'
GROUP BY b.flight_id
ORDER BY revenue DESC;
```

JOIN bookings to payments, then GROUP BY flight_id.

Q3. Find total revenue broken down by seat class.

Answer:

```
SELECT s.seat_class, SUM(p.amount) AS revenue
FROM payments p
JOIN bookings b ON b.booking_id = p.booking_id
JOIN seats s ON s.seat_id = b.seat_id
WHERE p.payment_status = 'Completed'
GROUP BY s.seat_class
ORDER BY revenue DESC;
```

Two joins connect payments back to the seat's class.

Q4. Find total revenue broken down by payment method.

Answer:

```
SELECT payment_method, SUM(amount) AS revenue, COUNT(*) AS transactions
FROM payments
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY revenue DESC;
```

GROUP BY directly on the payments table.

Q5. Find the monthly revenue trend.

Answer:

```
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS month, SUM(amount) AS revenue
FROM payments
WHERE payment_status = 'Completed'
GROUP BY month
ORDER BY month;
```

DATE_FORMAT buckets payments by month.

Q6. Find the top 5 revenue-generating flights.

Answer:

```
SELECT b.flight_id, SUM(p.amount) AS revenue
FROM bookings b
JOIN payments p ON p.booking_id = b.booking_id
WHERE p.payment_status = 'Completed'
GROUP BY b.flight_id
ORDER BY revenue DESC
LIMIT 5;
```

Builds on the per-flight revenue query with LIMIT 5.

Q7. Find the top 5 revenue-generating routes using a CTE.

Answer:

```
WITH route_revenue AS (  
  SELECT src.city AS source_city, dst.city AS destination_city, p.amount  
  FROM payments p  
  JOIN bookings b ON b.booking_id = p.booking_id  
  JOIN flights f ON f.flight_id = b.flight_id  
  JOIN airports src ON src.airport_id = f.source_airport_id  
  JOIN airports dst ON dst.airport_id = f.destination_airport_id  
  WHERE p.payment_status = 'Completed'  
)  
SELECT source_city, destination_city, SUM(amount) AS total_revenue  
FROM route_revenue  
GROUP BY source_city, destination_city  
ORDER BY total_revenue DESC  
LIMIT 5;
```

A WITH clause pre-joins the route data once, then the outer query aggregates it.

Q8. Find the average revenue per booking.

Answer:

```
SELECT ROUND(AVG(amount), 2) AS avg_revenue_per_booking  
FROM payments  
WHERE payment_status = 'Completed';
```

Simple AVG() over completed payments.

Q9. Find total revenue broken down by source city.

Answer:

```
SELECT src.city AS source_city, SUM(p.amount) AS revenue  
FROM payments p  
JOIN bookings b ON b.booking_id = p.booking_id  
JOIN flights f ON f.flight_id = b.flight_id  
JOIN airports src ON src.airport_id = f.source_airport_id  
WHERE p.payment_status = 'Completed'  
GROUP BY src.city  
ORDER BY revenue DESC;
```

Joins through to the source airport's city before grouping.

Q10. Find total revenue broken down by destination city.

Answer:

```
SELECT dst.city AS destination_city, SUM(p.amount) AS revenue  
FROM payments p  
JOIN bookings b ON b.booking_id = p.booking_id  
JOIN flights f ON f.flight_id = b.flight_id  
JOIN airports dst ON dst.airport_id = f.destination_airport_id  
WHERE p.payment_status = 'Completed'  
GROUP BY dst.city  
ORDER BY revenue DESC;
```

Mirror of the previous query using the destination airport instead.

Q11. Find the total amount currently sitting in Pending payment status.

Answer:

```
SELECT SUM(amount) AS total_pending  
FROM payments  
WHERE payment_status = 'Pending';
```

SUM() filtered to Pending payments.

Q12. Find the total amount lost to Failed payments.

Answer:

```
SELECT SUM(amount) AS total_failed
FROM payments
WHERE payment_status = 'Failed';
```

Same pattern for the Failed status.

Q13. Find month-over-month revenue growth percentage.

Answer:

```
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
       ROUND(
         (revenue - LAG(revenue) OVER (ORDER BY month))
         / LAG(revenue) OVER (ORDER BY month) * 100, 2
       ) AS growth_pct
FROM (
  SELECT DATE_FORMAT(payment_date, '%Y-%m') AS month, SUM(amount) AS revenue
  FROM payments
  WHERE payment_status = 'Completed'
  GROUP BY month
) monthly
ORDER BY month;
```

LAG() looks back one row (the previous month) within the ordered window.

Q14. Find the cumulative (running total) revenue over time.

Answer:

```
SELECT month, revenue,
       SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue
FROM (
  SELECT DATE_FORMAT(payment_date, '%Y-%m') AS month, SUM(amount) AS revenue
  FROM payments
  WHERE payment_status = 'Completed'
  GROUP BY month
) monthly
ORDER BY month;
```

A window SUM() with an implicit running frame produces a running total.

Q15. Find how much revenue comes from the top 10% of customers by spend.

Answer:

```
WITH customer_spend AS (
  SELECT b.customer_id, SUM(p.amount) AS total_spent
  FROM bookings b
  JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
  GROUP BY b.customer_id
),
ranked AS (
  SELECT customer_id, total_spent,
         NTILE(10) OVER (ORDER BY total_spent DESC) AS decile
  FROM customer_spend
)
SELECT SUM(total_spent) AS top_decile_revenue
FROM ranked
WHERE decile = 1;
```

NTILE(10) splits customers into 10 equal-sized buckets ordered by spend; decile 1 is the top 10%.

Q16. Rank flights by revenue using DENSE_RANK().

Answer:

```
SELECT flight_id, revenue,
       DENSE_RANK() OVER (ORDER BY revenue DESC) AS revenue_rank
FROM (
  SELECT b.flight_id, SUM(p.amount) AS revenue
  FROM bookings b
  JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
  GROUP BY b.flight_id
) per_flight;
```

DENSE_RANK() leaves no gaps in the ranking sequence when values tie.

Q17. Find total revenue broken down by day of the week.

Answer:

```
SELECT DAYNAME(payment_date) AS day_of_week, SUM(amount) AS revenue
FROM payments
WHERE payment_status = 'Completed'
GROUP BY day_of_week
ORDER BY revenue DESC;
```

DAYNAME() + GROUP BY on the payment date.

Q18. Find the average transaction value for each payment method.

Answer:

```
SELECT payment_method, ROUND(AVG(amount), 2) AS avg_transaction_value
FROM payments
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY avg_transaction_value DESC;
```

AVG() grouped by payment_method.

Q19. Find what fraction of gross bookings value was ultimately refunded.

Answer:

```
SELECT
  (SELECT SUM(amount) FROM payments
   WHERE payment_status = 'Refunded') AS total_refunded,
  (SELECT SUM(amount) FROM payments
   WHERE payment_status IN ('Completed', 'Refunded')) AS gross_value,
  ROUND(
    (SELECT SUM(amount) FROM payments WHERE payment_status = 'Refunded')
    / (SELECT SUM(amount) FROM payments
       WHERE payment_status IN ('Completed', 'Refunded')) * 100, 2
  ) AS refund_pct;
```

Scalar subqueries build a small refund-vs-gross summary in one row.

Q20. Find total revenue broken down by year.

Answer:

```
SELECT YEAR(payment_date) AS year, SUM(amount) AS revenue
FROM payments
WHERE payment_status = 'Completed'
GROUP BY year
ORDER BY year;
```

YEAR() extracts the calendar year for grouping.

Q21. Find the single highest-value payment transaction.

Answer:

```
SELECT payment_id, booking_id, amount, payment_date
FROM payments
ORDER BY amount DESC
LIMIT 1;
```

Simple ORDER BY + LIMIT 1.

Q22. Find total revenue broken down by quarter.

Answer:

```
SELECT CONCAT(YEAR(payment_date), '-Q', QUARTER(payment_date)) AS quarter, SUM(amount) AS revenue
FROM payments
WHERE payment_status = 'Completed'
GROUP BY quarter
ORDER BY quarter;
```

QUARTER() returns 1-4; combined with YEAR() for a readable label.

Q23. Find the percentage revenue contribution of each seat class to total revenue.

Answer:

```
SELECT s.seat_class, SUM(p.amount) AS revenue,
       ROUND(SUM(p.amount) * 100.0 /
             (SELECT SUM(amount) FROM payments
              WHERE payment_status = 'Completed'), 2) AS pct_of_total
FROM payments p
JOIN bookings b ON b.booking_id = p.booking_id
JOIN seats s ON s.seat_id = b.seat_id
WHERE p.payment_status = 'Completed'
GROUP BY s.seat_class
ORDER BY pct_of_total DESC;
```

A scalar subquery supplies the grand total as the percentage denominator.

Q24. Using a CTE, find flights whose revenue is above the average flight revenue.

Answer:

```
WITH flight_revenue AS (
  SELECT b.flight_id, SUM(p.amount) AS revenue
  FROM bookings b
  JOIN payments p ON p.booking_id = b.booking_id AND p.payment_status = 'Completed'
  GROUP BY b.flight_id
)
SELECT flight_id, revenue
FROM flight_revenue
WHERE revenue > (SELECT AVG(revenue) FROM flight_revenue)
ORDER BY revenue DESC;
```

The CTE is reused twice: once for the row set, once inside the subquery for the average.

Q25. Using vw_flight_revenue, list flights with revenue above 50,000.

Answer:

```
SELECT flight_id, flight_number, total_revenue
FROM vw_flight_revenue
WHERE total_revenue > 50000
ORDER BY total_revenue DESC;
```

Queries directly against the pre-built vw_flight_revenue view.